بخش اول: base.py

۱. import ها و ثابت

```
import math
import random
from abc import ABC, abstractmethod
TOL = 1e-9
```

- ABC و abstractmethod کلاس را انتزاعی می‌کنند. یعنی نمی‌شود از خود LocalSearchSolver شی ساخت و هر الگوریتم مجبور است solve خودش را بنویسد.
- TOL تلورانس مقایسه است. هزینه‌ها اعداد اعشاری‌اند (مثلاً 31.98000000001 و 31.98)، و بدون تلورانس این دو «متفاوت» حساب می‌شوند. آن وقت الگوریتم فکر می‌کند بهتر شده.

```python
class LocalSearchSolver(ABC):
    def __init__(self, problem: CommuteProblem, **kwargs):
        self.problem = problem
        self.params: Dict[str, Any] = kwargs
        self.history: List[float] = []
        self.rng = random.Random(kwargs.get("seed"))

        self.best_state: Optional[State] = None
        self.best_cost: float = math.inf
        self.best_step: int = 0
        self._costs: Dict[State, float] = {}
```

problem شیء مسئله: گراف، مدل ترافیک، مبدأ، مقصد و بازه‌ی زمانی
params پارامترهای config.json که main.py با **kwargs می‌فرستد
history بهترین هزینه در هر گام، برای نمودار همگرایی (main.py همین را می‌خواند)
rng مولد تصادفی اختصاصی. اگر seed داده شود، اجرا تکرارپذیر است
best_state, best_cost بهترین جوابی که تا الان پیدا شده
best_step در چه گامی بهترین جواب پیدا شد
_costs: cache هزینه ها

```python
def f(self, state: State) -> float:
    if state not in self._costs:
        self._costs[state] = self.problem.evaluate(state)
    return self._costs[state]

def consider(self, state: State, cost: float) -> None:
    if cost < self.best_cost - TOL:
        self.best_state, self.best_cost = state, cost
        self.best_step = len(self.history)

def log_step(self) -> None:
    self.history.append(self.best_cost)
```

متد f(state)

- evaluate برای هر یال مسیر، زمان ورود را حساب می‌کند و ضریب ترافیک گوسی را اعمال می‌کند.
- الگوریتم‌های محلی یک حالت را بارها می‌بینند. مثلا در Hill Climbing، حالت قبلی همسایه‌ی حالت فعلی هم هست.
- چون @dataclass(frozen=True) است و hash دارد State می‌تواند کلید دیکشنری باشد .
- evaluate قطعی (deterministic) است: یک (P, t) همیشه یک هزینه دارد cache کردن مجاز است.
- len(self._costs) تعداد ارزیابی‌های یکتا را نشان می‌دهد، که معیار خوبی برای مقایسه‌ی هزینه‌ی الگوریتم‌هاست.

متد consider(state, cost)

- فقط اگر هزینه واقعاً کمتر باشد (حداقل TOL کمتر)، بهترین جواب عوض می‌شود.
- len(self.history) همان شماره‌ی گام فعلی است، چون در هر گام یک عضو به history اضافه می‌شود.

متد log_step()

در پایان هر گام، بهترین هزینه تا این لحظه ثبت می‌شود، نه هزینه‌ی حالت فعلی. به همین دلیل نمودار همگرایی همیشه نزولی یا صاف است، حتی برای SA که گاهی حالت بدتر را می‌پذیرد.

solve انتزاعی

هر الگوریتم باید آن را پیاده کند و خروجی آن (best_state, best_cost) است.

```python
@abstractmethod
def solve(self) -> Tuple[State, float]:
    """
    Executes the search procedure.

    Returns:
        Tuple[State, float]: The best State found and its evaluated objective cost.
    """
    pass
```

هر چهار الگوریتم به یک سؤال مشترک نیاز دارند: این حالت به کجا می‌شود رفت؟ به جای اینکه هر الگوریتم این را جدا بنویسد، همه‌ی عملگرها در یک فایل هستند. این فایل ساختار فضای حالت را تعریف می‌کند، یعنی کدام حالت‌ها کنار هم هستند. الگوریتم‌ها فقط تعیین می‌کنند کدام همسایه را انتخاب کنند.
شرط اصلی صورت سوال این است که هیچ حالت خروجی‌ای دور، گسستگی یا زمانِ بیرون از بازه نداشته باشد. همه‌ی این قوانین فقط در همین فایل operators.py کنترل می‌شوند.

بخش دوم: operators.py

```
Path = Tuple[str, ...]
SEGMENT_LEN = 3
DETOUR_LEN = 4
```

- Path فقط یک نام کوتاه برای نوع تاپلی از نام گره‌ها است.
- SEGMENT_LEN: حداکثر طول (تعداد یال) تکه‌ای از مسیر که در یک حرکت عوض می‌شود.
- DETOUR_LEN: حداکثر طول تکه‌ی جدیدی که جایگزین آن می‌شود.
- این دو عدد شعاع همسایگی را تعیین می‌کنند. اگر کوچک باشند، همسایگی کوچک است و حرکت‌ها سریع‌اند، ولی کمینه‌ی محلی زیاد است. اگر بزرگ باشند، برعکس

۱. shift_time — حرکت زمانی

```python
def shift_time(problem: CommuteProblem, t: float, steps: int) -> Optional[float]:
    times = problem.get_valid_time_domain()
    idx = min(range(len(times)), key=lambda k: abs(times[k] - t))
    idx += steps
    return times[idx] if 0 <= idx < len(times) else None
```

-لیست همه‌ی زمان‌های مجاز را می‌گیرد: از ۷:۰۰ تا ۱۰:۰۰ با گام ۵ دقیقه، یعنی ۳۷ زمان.
-اندیس نزدیک‌ترین زمان به t را پیدا می‌کند. از «نزدیک‌ترین» استفاده می‌کنیم، نه index(t)، چون اعداد اعشاری ممکن است کمی خطا داشته باشند.
-steps را به اندیس اضافه می‌کند (۱- یعنی ۵ دقیقه زودتر، ۱+ یعنی ۵ دقیقه دیرتر).
-اگر اندیس بیرون از لیست باشد، None برمی‌گرداند و این حرکت ساخته نمی‌شود.
پس زمان هیچ‌وقت از [t_min, t_max] بیرون نمی‌رود و همیشه روی شبکه‌ی Δt است، چون مستقیما از لیست زمان‌های مجاز برداشته می‌شود.

۲. mix_times — زمان فرزند در crossover

```python
def mix_times(problem: CommuteProblem, t1: float, t2: float, rng) -> float:
    times = problem.get_valid_time_domain()
    mid = (t1 + t2) / 2.0
    lower = [t for t in times if t <= mid + 1e-9]
    upper = [t for t in times if t >= mid - 1e-9]
    options = {lower[-1] if lower else times[0], upper[0] if upper else times[-1]}
    return rng.choice(sorted(options))
```

- میانگین زمان دو والد ممکن است روی شبکه نباشد.
- lower[-1] بزرگ‌ترین زمان مجاز کوچک‌تر یا مساوی میانگین است (۷:۰۵) و upper[0] کوچک‌ترین زمان مجاز بزرگ‌تر یا مساوی میانگین.
- اگر میانگین دقیقا روی شبکه باشد، هر دو یکی می‌شوند و چون options یک set است، فقط یک گزینه می‌ماند.
- در غیر این صورت یکی از دو گزینه تصادفی انتخاب می‌شود. این کمی تنوع به جمعیت اضافه می‌کند.
- sorted برای این است که ترتیب set ثابت باشد و با seed یکسان، نتیجه هم یکسان باشد.

detours .۳

```
def detours(problem: CommuteProblem, a: str, b: str,
            blocked: Set[str], limit: int) -> List[Path]:
    found: List[Path] = []

    def extend(walk: List[str]) -> None:
        last = walk[-1]
        if last == b:
            found.append(tuple(walk))
            return
        if len(walk) - 1 == limit:
            return
        for nxt in problem.network.get_neighbors(last):
            if nxt in blocked or nxt in walk:
                continue
            walk.append(nxt)
            extend(walk)
            walk.pop()

    extend([a])
    return found
```

این یک DFS بازگشتی با backtracking است:
- walk مسیری است که تا الان ساخته شده (از a شروع می‌شود).
- اگر آخرین گره b باشد، یک تکه‌ی کامل پیدا شده و ذخیره می‌شود.
- len(walk) - 1 تعداد یال‌هاست. اگر به limit برسد، دیگر جلو نمی‌رویم.
- گره بعدی دو شرط دارد:
  - nxt in blocked: نباید گره‌ای باشد که در بقیه‌ی مسیر اصلی است. این کلید اصلی بدون دور بودن است.
  - nxt in walk: نباید داخل خود تکه تکراری باشد.
- بعد از برگشت از بازگشت، walk.pop() گره را برمی‌دارد تا شاخه‌ی بعدی امتحان شود.
- فقط از network.get_neighbors جلو می‌رویم، پس هر یال واقعا در گراف وجود دارد.

۴. connect

```python
def connect(problem: CommuteProblem, a: str, b: str,
            blocked: Set[str], limit: int) -> Optional[Path]:
    prev = {a: None}
    frontier = deque([(a, 0)])
    while frontier:
        node, depth = frontier.popleft()
        if node == b:
            walk = []
            while node is not None:
                walk.append(node)
                node = prev[node]
            return tuple(reversed(walk))
        if depth == limit:
            continue
        for nxt in problem.network.get_neighbors(node):
            if nxt not in prev and (nxt == b or nxt not in blocked):
                prev[nxt] = node
                frontier.append((nxt, depth + 1))
    return None
```

- BFS سطح به سطح جلو می‌رود، پس اولین باری که به b برسد، کوتاه‌ترین مسیر (کمترین تعداد یال) را پیدا کرده است.
- prev دو کار می‌کند: گره‌های دیده‌شده را علامت می‌زند تا دوباره دیده نشوند، و پدر هر گره را نگه می‌دارد.
- بازسازی مسیر: از b شروع می‌کنیم و با prev به عقب برمی‌گردیم تا به a برسیم، بعد مسیر را برعکس می‌کنیم.
- شرط (nxt == b or nxt not in blocked): گره‌های ممنوعه رد می‌شوند، به جز خود b. چون b جزو دم مسیر است و حتماً در blocked قرار دارد، بدون این استثنا هیچ‌وقت به آن نمی‌رسیدیم.
- limit طول پل را محدود می‌کند. اگر پلی پیدا نشود، None برمی‌گرداند.

فرق detours و connect: detours همه‌ی مسیرها را می‌خواهد (برای همسایگی) و از DFS استفاده می‌کند. connect فقط یکی، کوتاه‌ترینش را می‌خواهد (برای پل crossover) و از BFS استفاده می‌کند.

۵.

```python
def is_simple(path: Path) -> bool:
    return len(path) == len(set(path))
```

اگر گره تکراری وجود داشته باشد، set کوچک‌تر از لیست می‌شود، یعنی مسیر دور دارد.

۶. get_neighbors

قسمت ۱، حرکت زمانی:

```
    Returns:
        List[State]: A list of valid neighboring State objects.
    """
    path, t = state.path, state.departure_time
    candidates: List[State] = []
    for step in (-1, +1):
        new_t = shift_time(problem, t, step)
        if new_t is not None:
            candidates.append(State(path, new_t))
```

همان مسیر با ۵ دقیقه زودتر یا دیرتر. در لبه‌ی بازه فقط یکی از این دو ساخته می‌شود.

قسمت ۲، حرکت مسیری:

```
last = len(path) - 1
for i in range(last):
    for j in range(i + 1, min(last, i + SEGMENT_LEN) + 1):
        keep = set(path[:i]) | set(path[j + 1:])
        for piece in detours(problem, path[i], path[j], keep, DETOUR_LEN):
            if piece != path[i:j + 1]:
                candidates.append(State(path[:i] + piece + path[j + 1:], t))
```

- i شروع تکه و j پایان آن است.
- keep گره‌هایی است که بیرون از تکه هستند: هر چیزی قبل از i و بعد از j. خود path[i] و path[j] جزو keep نیستند، چون تکه‌ی جدید باید از آن‌ها شروع شود و به آن‌ها ختم شود.
- piece != path[i:j+1]: تکه‌ای که همان تکه‌ی قبلی باشد، همسایه نیست.
- مسیر جدید از سه قسمت ساخته می‌شود: path[:i] (سرِ دست‌نخورده) + piece (تکه‌ی جدید) + path[j+1:] (دم دست‌نخورده).

سر و دم از مسیر اصلی می‌آیند که بدون دور بود، پس با هم تداخل ندارند. تکه‌ی جدید وارد هیچ گره‌ای از سر و دُم نمی‌شود (keep)، و داخل خودش هم تکرار ندارد (nxt in walk). پس هیچ گره‌ای دو بار نمی‌آید -> دور نداریم

چرا پیوسته است؟ سر به path[i] ختم می‌شود و تکه از همان path[i] شروع می‌شود. تکه به path[j] ختم می‌شود و دُم از path[j+1] ادامه پیدا می‌کند، که یال (path[j], path[j+1]) در مسیر اصلی بود.
همه‌ی یال‌های تکه هم از گراف آمده‌اند.

قسمت ۳:

```
    result, seen = [], {state}
    for c in candidates:
        if c not in seen and problem.is_valid_state(c):
            seen.add(c)
            result.append(c)
    return result
```

- seen از ابتدا خود state را دارد
- تکراری‌ها حذف می‌شوند. ممکن است دو جفت (i, j) متفاوت به یک مسیر یکسان برسند.
- is_valid_state یک لایه‌ی امنیتی اخر است.

۷. crossover

```
    rng = rng or random
    a, b = parent1.path, parent2.path
    t = mix_times(problem, parent1.departure_time, parent2.departure_time, rng)
```

اگر rng داده نشود، از random معمولی پایتون استفاده می‌شود.

```
    State: A newly generated child State.
"""
rng = rng or random
a, b = parent1.path, parent2.path
t = mix_times(problem, parent1.departure_time, parent2.departure_time, rng)

shared = [n for n in a[1:-1] if n in b[1:-1]]
rng.shuffle(shared)
for node in shared:
    i, j = a.index(node), b.index(node)
    options = [a[:i] + b[j:], b[:j] + a[i:]]
    rng.shuffle(options)
    for child in options:
        if child not in (a, b) and is_simple(child) and problem.is_valid_path(child):
            return State(child, t)
```

- a[1:-1] یعنی گره‌های میانی، بدون Home و Uni (آن دو همیشه مشترک‌اند و برش در آن‌ها بی‌معنی است).
- اگر گره c در هر دو مسیر باشد:
- a[:i] سر a تا قبل از c است و b[j:] دم b از خود c به بعد. پس c فقط یک بار می‌آید و پیوستگی حفظ می‌شود.
- گزینه‌ی دوم برعکس است.
- شرط‌های قبول فرزند:
- child not in (a, b): فرزند نباید کپی یکی از والدها باشد.
- is_simple(child): ممکن است گره‌ای هم در سر a و هم در دم b باشد که دور بسازد. این شرط آن را رد می‌کند.
- is_valid_path(child): بررسی نهایی.

مرحله‌ی ۲، وقتی مرحله‌ی ۱ فرزندی نساخت، مثلاً وقتی گره مشترکی وجود ندارد

```
    cuts = [(i, j) for i in range(1, len(a) - 1) for j in range(1, len(b) - 1)]
    rng.shuffle(cuts)
    for i, j in cuts:
        head, tail = a[:i + 1], b[j:]
        if set(head) & set(tail):
            continue
        bridge = connect(problem, a[i], b[j], set(head) | set(tail), DETOUR_LEN)
        if bridge is None:
            continue
        child = head[:-1] + bridge + tail[1:]
        if is_simple(child) and problem.is_valid_path(child):
            return State(child, t)
```

- همه‌ی جفت نقطه‌ی برش (i در a، j در b) امتحان می‌شوند، به ترتیب تصادفی.
- head از Home تا a[i] است (بخشی از والد اول) و tail از b[j] تا Uni (بخشی از والد دوم).
- اگر head و tail گره مشترک داشته باشند، دور ایجاد می‌شود، پس این برش رد می‌شود.
- connect از a[i] به b[j] یک پل پیدا می‌کند که از هیچ گره‌ای در head یا tail نمی‌گذرد.
- head[:-1] + bridge + tail[1:]: پل با a[i] شروع می‌شود و با b[j] تمام می‌شود، پس آخرین گره‌ی head و اولین گره‌ی tail حذف می‌شوند تا تکرار نشوند.
- این پاسخ پرسش ۶ است وقتی دو مسیر کاملا جدا هستند، crossover کلاسیک ممکن نیست. ما سر یکی را به دم دیگری با یک پل معتبر وصل می‌کنیم.

مرحله‌ی ۳، آخرین راه:

```
    base = State(rng.choice((a, b)), t)
    moves = get_neighbors(base, problem)
    return rng.choice(moves) if moves else base
```

اگر هیچ پلی هم پیدا نشود، یکی از والدها انتخاب می‌شود و یک جهش روی آن اعمال می‌شود. همسایه‌ها همیشه معتبرند، پس خروجی crossover در هیچ حالتی نامعتبر نیست.

random_state .۸

```python
def random_state(problem: CommuteProblem, rng: Optional[random.Random] = None) -> State:
    """A random loop-free route (randomised backtracking DFS) with a random departure slot."""
    rng = rng or random
    goal = problem.destination

    def walk(route: List[str]) -> Optional[Path]:
        if route[-1] == goal:
            return tuple(route)
        nexts = [n for n in problem.network.get_neighbors(route[-1]) if n not in route]
        rng.shuffle(nexts)
        for n in nexts:
            route.append(n)
            done = walk(route)
            if done:
                return done
            route.pop()
        return None

    route = walk([problem.origin])
    if route is None:
        return problem.generate_initial_state()
    return State(route, rng.choice(problem.get_valid_time_domain()))

```

- مثل detours یک DFS بازگشتی است، ولی با دو فرق:
- همسایه‌ها به هم ریخته می‌شوند، پس هر بار مسیر متفاوتی پیدا می‌شود.
- با پیدا شدن اولین مسیر کامل متوقف می‌شود.
- اگر از یک گره به بن‌بست برسیم، route.pop() آن را برمی‌دارد و گزینه‌ی بعدی امتحان می‌شود (backtracking). پس اگر مسیری وجود داشته باشد، حتماً پیدا می‌شود.
- n not in route: بدون دور.
- ساعت حرکت به صورت یکنواخت از بین زمان‌های مجاز انتخاب می‌شود.
- اگر هیچ مسیری نباشد، حالت پیش‌فرض مسئله برگردانده می‌شود.

نقطه‌ی شروع جدید برای random restart در HC، و ساختن K پرتو اولیه در Beam و جمعیت اولیه در GA. هرچه این حالت‌ها متنوع‌تر باشند، بخش بیشتری از فضای حالت پوشش داده می‌شود.

الگوریتمHill Climbing

همیشه به بهترین همسایه برو. اگر هیچ همسایه‌ای بهتر نبود، متوقف شو

```python
    restart = 0
    while restart <= self.max_restarts:
        if restart == 0:
            current = self.problem.generate_initial_state() # first climb is from default state
        else:
            current =random_state(self.problem, self.rng) #  others random
        current_cost = self.f(current)
        self.consider(current, current_cost)
        self.log_step()

        sideways = 0
        visited = {current}
        while True:
            neighbors = get_neighbors(current, self.problem)
            if not neighbors:
                break

            # steepest descent
            best_cost = min(self.f(n) for n in neighbors)
            best = [n for n in neighbors if self.f(n) <= best_cost + TOL]

            if best_cost < current_cost - TOL:
                # strictly better
                sideways = 0
```

```
restart = 0
while restart <= self.max_restarts:
    if restart == 0:
        current = self.problem.generate_initial_state() # first climb is from default state
    else:
        current =random_state(self.problem, self.rng) #  others random
    current_cost = self.f(current)
    self.consider(current, current_cost)
    self.log_step()

    sideways = 0
    visited = {current}
    while True:
        neighbors = get_neighbors(current, self.problem)
        if not neighbors:
            break

        # steepest descent
        best_cost = min(self.f(n) for n in neighbors)
        best = [n for n in neighbors if self.f(n) <= best_cost + TOL]

        if best_cost < current_cost - TOL:
            # strictly better
            sideways = 0
```

همه‌ی همسایه‌ها بررسی می‌شوند و کم‌هزینه‌ترین‌ها انتخاب می‌شوند. این همان Steepest Descent است. اگر چند همسایه با بهترین هزینه هم‌قیمت باشند، همه در best می‌روند و بعدا یکی تصادفی انتخاب می‌شود.

چرا حرکت افقی؟ روی فلات همه‌ی همسایه‌ها هزینه‌ی برابر دارند. بدون حرکت افقی، الگوریتم فورا متوقف می‌شود، در حالی که شاید چند قدم آن طرف‌تر یک سراشیبی باشد.

چرا سقف max_sideways؟ بدون سقف، الگوریتم روی فلات تا ابد چپ و راست می‌رود.
چرا visited؟ روی فلات ممکن است بین دو حالت A و B مدام رفت و برگشت کنیم. با ترجیح دادن حالت‌های دیده‌نشده، الگوریتم واقعا روی فلات جلو می‌رود.

چرا sideways = 0 بعد از بهبود؟ وقتی پایین رفتیم، به یک ناحیه‌ی جدید رسیده‌ایم و سهمیه‌ی حرکت افقی دوباره پر می‌شود.

الگوریتم Simulated Annealing

در کد ما انرژی همان زمان سفر f(x) است.

فرق اصلی با HC ، HC هیچ‌وقت حرکت بد را قبول نمی‌کند، پس در کمینه‌ی محلی گیر می‌کند. SA گاهی حرکت بد را قبول می‌کند تا از کمینه‌ی محلی بیرون بپرد. هرچه دما پایین‌تر بیاید، کمتر این کار را می‌کند.

```
    """
    state = self.problem.generate_initial_state()
    energy = self.f(state)
    self.consider(state, energy)
    self.log_step()

    temp = self.initial_temp
    step = 0
    while step < self.max_iterations and temp >= self.min_temp:
        moves = get_neighbors(state, self.problem)
        if len(moves) == 0:
            break

        nxt = moves[self.rng.randrange(len(moves))]
        nxt_energy = self.f(nxt)

        if self.accept(nxt_energy - energy, temp):
            state, energy = nxt, nxt_energy
            self.consider(state, energy)

        self.log_step()
        temp *=self.cooling_rate
        step += 1

    return self.best_state, self.best_cost
```

از حالت پیش‌فرض شروع می‌کنیم، هزینه‌اش را حساب می‌کنیم و آن را به عنوان اولین «بهترین» ثبت می‌کنیم.

حلقه‌ی اصلی دو شرط توقف در یک خط:
- تعداد گام از سقف بیشتر نشود.
- دما از حداقل پایین‌تر نرود. وقتی دما خیلی کم است، تقریبا هیچ حرکت بدی پذیرفته نمی‌شود
- انتخاب یک همسایه‌ی تصادفی
- فقط یک همسایه به صورت تصادفی انتخاب می‌شود (HC همه را بررسی می‌کرد).
- randrange(len(moves)) یک اندیس تصادفی بین ۰ و تعداد همسایه‌ها منهای یک است.
- self.rng همان مولد تصادفی با seed است، پس اجرا قابل تکرار است.

-تصمیم پذیرش

```python
def accept(self, delta: float, temp: float) -> bool:
    #Metropolis rule
    if delta <= 0:
        return True
    elif self.rng.random() < math.exp(-delta / temp):
        self.uphill_moves += 1
        return True
    else:
        return False
```

معنی فرمول P = e^(−ΔE/T) (معیار Metropolis):
- random() یک عدد تصادفی بین ۰ و ۱ است. اگر از P کمتر باشد، حرکت پذیرفته می‌شود، پس احتمال پذیرش دقیقا P است.
- اگر T بزرگ باشد، P نزدیک ۱ است و حرکت‌های بد هم پذیرفته می‌شوند.
- اگر T کوچک باشد، P نزدیک ۰ است و فقط حرکت‌های خوب پذیرفته می‌شوند (بهره‌برداری، مثل HC).
- اگر ΔE بزرگ باشد (خیلی بدتر)، P کوچکتر است، یعنی حرکت‌های کمی بد راحت‌تر از خیلی بد پذیرفته می‌شوند.

-بعد از پذیرش
حالت فعلی عوض می‌شود و اگر بهترین تا الان باشد، ذخیره می‌شود.

-سرد کردن
برنامه‌ی سرد کردن هندسی است: $T_{(k+1)} = \alpha \cdot T_{(k)}$.

الگوریتمLocal Beam Search

به جای یک حالت، K حالت همزمان نگه می‌داریم، K = beam_width = 8. در هر دور همسایه‌های همه‌ی آن‌ها در یک استخر مشترک ریخته می‌شوند و K تای برتر کل استخر انتخاب می‌شوند.

فرق با K بار HC جداگانه: اگر یکی از پرتوها به ناحیه‌ی خوبی برسد، همسایه‌هایش جای پرتوهای بد را می‌گیرند. پس تلاش جستجو به سمت ناحیه‌ی امیدوارکننده متمرکز می‌شود.

```
    # the default state plus random ones , no duplicates
    beam = [self.problem.generate_initial_state()]
    tries = 0
    while len(beam) < self.beam_width and tries < 100:
        s = random_state(self.problem, self.rng)
        if s not in beam:
            beam.append(s)
        tries += 1

    for s in beam:
        self.consider(s, self.f(s))
    self.log_step()

    no_progress = 0
    it = 0
    while it < self.max_iterations and no_progress < self.patience:
        pool = set()
        for s in beam:
            for n in get_neighbors(s, self.problem):
                pool.add(n)
        if len(pool) == 0:
            break

        ranked = sorted(pool, key=lambda s: (self.f(s), s.departure_time, s.path))
```

```
        beam = []
        route_count = {}
        for s in ranked:
            if route_count.get(s.path, 0) >= self.same_route_limit:
                continue
            beam.append(s)
            route_count[s.path] = route_count.get(s.path, 0) + 1
            if len(beam) == self.beam_width:
                break

        old_best = self.best_cost
        self.consider(beam[0], self.f(beam[0]))
        if self.best_cost < old_best - TOL:
            no_progress = 0
        else:
            no_progress += 1

        self.log_step()
        it += 1

    return self.best_state, self.best_cost
```

- same_route_limit = 2: حداکثر ۲ پرتو می‌توانند مسیر یکسان داشته باشند این برای حفظ تنوع است.
- patience = 10: اگر ۱۰ دور پشت سر هم بهترین جواب بهتر نشود، الگوریتم متوقف می‌شود.

-پرتوهای اولیه
یکی حالت پیش‌فرض است و بقیه تصادفی و غیرتکراری. tries جلوی حلقه‌ی بی‌پایان را می‌گیرد، اگر حالت‌های متفاوت کافی وجود نداشته باشد.
:while it < self.max_iterations and no_progress < self.patience
یعنی تا وقتی گام‌ها تمام نشده و هنوز داریم پیشرفت می‌کنیم. این همان شرط صورت سوال است توقف وقتی پرتو دیگر بهتر نمی‌شود. set باعث می‌شود اگر دو پرتو همسایه‌ی مشترک داشته باشند، آن همسایه یک بار حساب شود.
مرتب‌سازی اول بر اساس هزینه است. اگر هزینه‌ها برابر باشند، بر اساس زمان و مسیر مرتب می‌شود، تا ترتیب همیشه ثابت باشد و نتایج قابل تکرار.
- انتخاب K تای برتر با حفظ تنوع
- بررسی پیشرفت:
beam[0] بهترین حالت پرتو جدید است. اگر رکورد را شکست، شمارنده صفر می‌شود. در غیر این صورت یکی زیاد می‌شود.

الگوریتم Genetic Algorithm

-تابع برازندگی (fitness)

```
def fitness(self, state: State) -> float:
    return 1.0 / (1.0 + self.f(state))
```

- ما می‌خواهیم زمان سفر کم باشد، ولی GA دنبال fitness زیاد است. پس باید معکوس کنیم.
- 1 + تا هیچ‌وقت تقسیم بر صفر نداشته باشیم.
- حالت نامعتبر (هزینه ∞) fitness صفر می‌گیرد.

- انتخاب والد، Tournament (pick_parent)

```
def pick_parent(self, population):
    group = self.rng.sample(population, self.tournament_size)
    winner = group[0]
    for s in group:
        if self.fitness(s) > self.fitness(winner):
            winner = s
    return winner
```

group = self.rng.sample(population, self.tournament_size) نفر تصادفی
winner = بهترینِ گروه
- چرا خوب است؟ بهترها بیشتر برنده می‌شوند، ولی ضعیف‌ها هم گاهی شانس دارند. این تنوع جمعیت را حفظ می‌کند.
- فشار انتخاب: tournament_size بزرگ‌تر یعنی فقط قوی‌ترین‌ها انتخاب می‌شوند. همگرایی سریع‌تر است، ولی خطر همگرایی زودرس وجود دارد.

- جهش (mutate)

```python
def mutate(self, state: State) -> State:
    options = get_neighbors(state, self.problem)
    if options:
        return self.rng.choice(options)
    return state
```

یک همسایه‌ی تصادفی انتخاب می‌شود، یعنی یک تغییر کوچک در زمان یا یک انحراف کوچک در مسیر. چون از get_neighbors استفاده می‌کند، جهش هیچ‌وقت حالت نامعتبر نمی‌سازد.

ادامه کد جمعیت اولیه یعد اول جمعیت را از بهترین به بدترین مرتب می‌کنیم. بهترین عضو ثبت می‌شود بعد ۳ عضو برتر بدون تغییر به نسل بعد می‌روند. چرا؟ crossover و جهش ممکن است جواب‌های خوب را خراب کنند. elitism تضمین می‌کند بهترین جواب هیچ‌وقت از دست نمی‌رود و بعد ساخت فرزندان

```python
    # the default state plus random ones , no duplicates
    beam = [self.problem.generate_initial_state()]
    tries = 0
    while len(beam) < self.beam_width and tries < 100:
        s = random_state(self.problem, self.rng)
        if s not in beam:
            beam.append(s)
        tries += 1

    for s in beam:
        self.consider(s, self.f(s))
    self.log_step()

    no_progress = 0
    it = 0
    while it < self.max_iterations and no_progress < self.patience:
        pool = set()
        for s in beam:
            for n in get_neighbors(s, self.problem):
                pool.add(n)
        if len(pool) == 0:
            break

        ranked = sorted(pool, key=lambda s: (self.f(s), s.departure_time, s.path))
```

```python
        beam = []
        route_count = {}
        for s in ranked:
            if route_count.get(s.path, 0) >= self.same_route_limit:
                continue
            beam.append(s)
            route_count[s.path] = route_count.get(s.path, 0) + 1
            if len(beam) == self.beam_width:
                break

        old_best = self.best_cost
        self.consider(beam[0], self.f(beam[0]))
        if self.best_cost < old_best - TOL:
            no_progress = 0
        else:
            no_progress += 1

        self.log_step()
        it += 1

    return self.best_state, self.best_cost
```

- همه‌ی اعداد با seed = 0 و تنظیمات نهایی config.json به دست آمده‌اند. آزمایش‌های تنظیم پارامتر با seedهای ۰ تا ۹ (۱۰ اجرا) انجام شده‌اند.

- **خلاصه و نحوه‌ی اجرا**

در این تمرین مسئله‌ی برنامه‌ریزی سفربا چهار الگوریتم جستجوی محلی حل شده است: Hill Climbing، Simulated Annealing، Local Beam Search و Genetic Algorithm. هر وضعیت یک تاپل ⟨P, t_start⟩ = x است. P یک مسیر ساده (بدون دور) از Home تا Uni است و t_start ساعت حرکت. تابع هدف f(x) کل زمان سفر است که با شبیه‌ساز ترافیک محاسبه می‌شود. زمان عبور از هر یال به لحظه‌ی ورود به آن بستگی دارد پس یک مسیر ثابت در ساعت‌های مختلف هزینه‌های متفاوتی دارد و مسیر و ساعت حرکت باید با هم بهینه شوند.
همه‌ی اجراها با seed ثابت انجام شده‌اند تا نتایج قابل تکرار باشند. برای این کار، مولد تصادفی هر الگوریتم (self.rng) از مولد سراسری‌ای مقدار می‌گیرد که main.py با --seed تنظیم می‌کند.

دستورهای اجرا:

```
python main.py --algo all --seed 0 --map data/maps/toy_graph.json --scenario data/scenarios/rush_hour.json
python main.py --algo all --seed 0 --plot-metrics --save-figs
python main.py --algo all --seed 0 --plot-route --save-figs
python main.py --interactive
```

هر سناریو روی نقشه‌ی مخصوص خودش اجرا شده است:

سناریو	نقشه	تعداد مسیر ساده	تعداد کل وضعیت‌ها
rush_hour	toy_graph	12	156
dual_peak_rush_hour	metropolis_graph	33	429
cascading_gridlock	metropolis_graph	33	429
crossover_showcase	metropolis_graph	33	429
flat_plateau	downtown_grid	10	130

۲. ابرپارامترها (config.json)

همه‌ی پارامترهای قابل تنظیم در فایل config.json قرار دارند. بازه‌ی زمانی مسئله از ۷:۴۵ تا ۸:۴۵ با گام ۵ دقیقه است، یعنی ۱۳ ساعت حرکت ممکن. در جدول زیر، مقدار قبلی داخل پرانتز آمده و جدید یعنی کلیدی که به فایل اضافه شده است.

الگوریتم	پارامتر	مقدار	
Hill Climbing	max_sideways	100	
Hill Climbing	max_restarts	10	
Simulated Annealing	initial_temp	120	
Simulated Annealing	cooling_rate	0.999 (0.992)	ضریب سرد شدن α در $T(k+1) = \alpha \cdot T(k)$
Simulated Annealing	min_temp	0.001	دمای توقف
Simulated Annealing	max_iterations	3000	
Local Beam Search	beam_width	8	تعداد وضعیت‌های موازی K
Local Beam Search	max_iterations	150	
Local Beam Search	same_route_limit	2 (جدید)	حداکثر تعداد پرتو با مسیر یکسان
Local Beam Search	patience	10 (جدید)	توقف بعد از این تعداد تکرار بدون بهبود
Genetic Algorithm	population_size	50	
Genetic Algorithm	generations	120	
Genetic Algorithm	crossover_rate	0.85	احتمال ترکیب دو والد
Genetic Algorithm	mutation_rate	0.25	
Genetic Algorithm	elitism	3	

Genetic Algorithm	tournament_size	3 (جدید) تعداد شرکت‌کنندگان در هر تورنمنت	

تنظیم Simulated Annealing:
با مقدار اولیه‌ی cooling_rate = 0.992، الگوریتم SA در سناریوی rush_hour فقط در ۹ اجرا از ۱۰ اجرا به جواب بهینه (حدود ۴۱ دقیقه) رسید. برای پیدا کردن مقدار بهتر، SA با ضریب‌های سرد شدن مختلف و ۱۰ seed اجرا شد:

T_0	α	موفقیت (از ۱۰)	میانگین زمان سفر حرکت بدتر پذیرفته‌شده			تعداد گام	زمان اجرا (ms)
120	0.5	0	48.19	2	17	1	
120	0.9	0	44.81	12	112	4	
120	0.992	9	41.94	180	1457	60	
120	0.999	10	41.77	1142	3000	127	

با α = 0.5 دما در هر گام نصف می‌شود و جستجو بعد از ۱۷ گام تمام می‌شود. در این مدت فقط ۲ حرکت بدتر پذیرفته شده و الگوریتم هیچ‌وقت به بهینه نرسیده است. با α = 0.999 دمای بالا مدت طولانی‌تری حفظ می‌شود، الگوریتم بیش از هزار حرکت بدتر را می‌پذیرد و در هر ۱۰ اجرا به بهینه می‌رسد. هزینه‌اش این است که زمان اجرا حدود دو برابر می‌شود (۱۲۷ میلی‌ثانیه در برابر ۶۰). چون کیفیت جواب برای ما مهم‌تر است و زمان اجرا هنوز کمتر از یک‌پنجم ثانیه است، α = 0.999 به‌عنوان مقدار نهایی انتخاب شد.

کلیدهای patience، same_route_limit و tournament_size هم به فایل اضافه شدند. قبلا کد از مقدار پیش‌فرض آن‌ها استفاده می‌کرد، ولی حالا بدون تغییر کد قابل تنظیم هستند.

۴. نتایج و جدول مقایسه‌ی الگوریتم‌ها

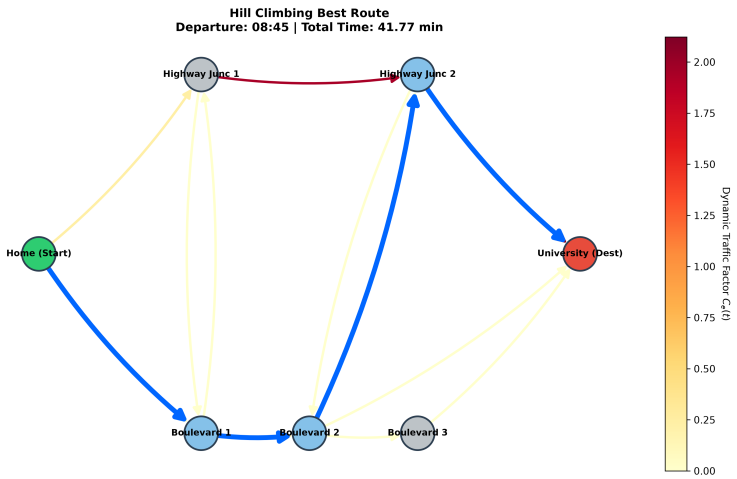
```
└─± python main.py --algo all --seed 0 --map data/maps/toy_graph.json --scenario data/scenarios/rush_hour.json
   Optimal Route    : Home -> B1 -> B2 -> H2 -> Uni

======================================================================
 BENCHMARK SUMMARY
======================================================================
 * Hill Climbing          :  41.77 min | Depart: 08:45 | Path: Home -> B1 -> B2 -> H2 -> Uni
 * Simulated Annealing    :  41.77 min | Depart: 08:45 | Path: Home -> B1 -> B2 -> H2 -> Uni
 * Local Beam Search      :  41.77 min | Depart: 08:45 | Path: Home -> B1 -> B2 -> H2 -> Uni
 * Genetic Algorithm      :  41.77 min | Depart: 08:45 | Path: Home -> B1 -> B2 -> H2 -> Uni
======================================================================

┌─sarahahmadi at sarahs-MacBook-Air in ~/Downloads/HW2_AI_403400002/commute-optimizer on mainxxx using «.venv»
```

همه‌ی اجراها با seed = 0 و تنظیمات نهایی config.json انجام شده‌اند. گام تا همگرایی یعنی شماره‌ی گامی که بهترین جواب نهایی برای اولین بار در آن پیدا شد، در کنار کل گام‌های اجرا. برای GA، هر گام یک نسل است.

سناریو	الگوریتم	بهترین زمان سفر f(x*) (دقیقه)	ساعت حرکت	مسیر	زمان اجرا (ms)	گام تا همگرایی / کل گام‌ها
rush_hour	Hill Climbing	41.77	08:45	Home → B1 → B2 → H2 → Uni	3	9 / 67
rush_hour	Simulated Annealing	41.77	08:45	Home → B1 → B2 → H2 → Uni	135	244 / 3000
rush_hour	Local Beam Search	41.77	08:45	Home → B1 → B2 → H2 → Uni	5	2 / 12
rush_hour	Genetic Algorithm	41.77	08:45	Home → B1 → B2 → H2 → Uni	139	2 / 120
dual_peak_rush_hour	Hill Climbing	31.69	08:45	Home → C1 → C2 → T2 → Uni	9	15 / 107
dual_peak_rush_hour	Simulated Annealing	31.69	08:45	Home → C1 → C2 → T2 → Uni	219	358 / 3000
dual_peak_rush_hour	Local Beam Search	31.69	08:45	Home → C1 → C2 → T2 → Uni	7	2 / 12
dual_peak_rush_hour	Genetic Algorithm	31.69	08:45	Home → C1 → C2 → T2 → Uni	181	8 / 120
cascading_gridlock	Hill Climbing	31.02	08:45	Home → C1 → C2 → B2 → Uni	87	12 / 1211
cascading_gridlock	Simulated Annealing	31.02	08:45	Home → C1 → C2 → T2 → Uni	216	1500 / 3000
cascading_gridlock	Local Beam Search	31.02	08:45	Home → C1 → C2 → T2 → Uni	7	2 / 12
cascading_gridlock	Genetic Algorithm	31.02	08:45	Home → C1 → C2 → T2 → Uni	176	12 / 120
crossover_showcase	Hill Climbing	31.01	08:45	Home → C1 → C2 → B2 → Uni	76	48 / 1038
crossover_showcase	Simulated Annealing	31.01	08:45	Home → C1 → C2 → B2 → Uni	215	1156 / 3000
crossover_showcase	Local Beam Search	31.01	08:45	Home → C1 → C2 → T2 → Uni	7	2 / 12
crossover_showcase	Genetic Algorithm	31.01	08:45	Home → C1 → C2 → B2 → Uni	174	8 / 120
flat_plateau	Hill Climbing	28.85	08:45	Home → G1 → G2 → G3 → G7 → Uni	5	13 / 93
flat_plateau	Simulated Annealing	28.85	08:45	Home → G1 → G2 → G3 → G7 → Uni	148	299 / 3000
flat_plateau	Local Beam Search	28.85	08:45	Home → G1 → G2 → G3 → G7 → Uni	6	4 / 14
flat_plateau	Genetic Algorithm	28.85	08:45	Home → G1 → G2 → G3 → G7 → Uni	162	1 / 120



تحلیل نتایج:
در هر پنج سناریو، هر چهار الگوریتم به یک زمان سفر رسیدند. ساعت حرکت همیشه ۸:۴۵ بود، یعنی آخرین ساعت مجاز. دلیلش این است که اوج ترافیک یال‌های مهم قبل از این ساعت است، پس دیرتر حرکت کردن یعنی دور شدن از شلوغی. مسیر بهینه‌ی rush_hour هم از بزرگراه شلوغ استفاده نمی‌کند. ابتدای بزرگراه را با B1→B2 دور می‌زند و فقط آخرین یال (H2→Uni) را، که اوجش در ۸:۲۵ گذشته، از بزرگراه می‌رود.

هزینه‌ی رسیدن به جواب: چون فضای حالت کوچک است، تفاوت الگوریتم‌ها در کیفیت جواب دیده نمی‌شود، بلکه در هزینه‌ی رسیدن به آن:

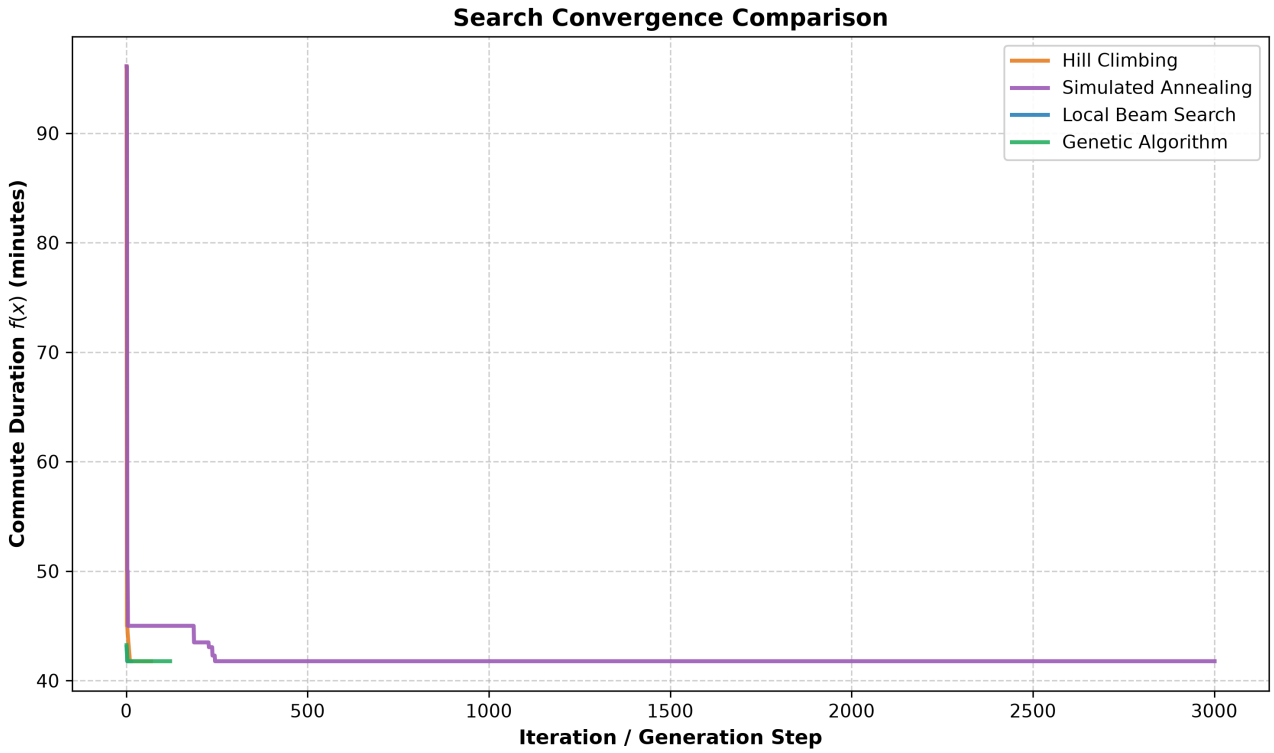
- Local Beam Search سریع‌ترین بود (۵ تا ۷ میلی‌ثانیه) و در همه‌ی سناریوها در ۲ تا ۴ گام به بهینه رسید. دلیلش این است که از ۸ نقطه‌ی شروع متفاوت آغاز می‌کند و همسایه‌های همه‌ی آن‌ها را مشترک مقایسه می‌کند، پس در هر گام بخش بزرگی از فضای حالت را می‌بیند. بعد از رسیدن به بهینه، به‌خاطر شرط patience فقط ۱۰ گام دیگر ادامه می‌دهد.

- Hill Climbing روی نقشه‌های کوچک خیلی سریع بود (۳ تا ۹ میلی‌ثانیه). روی cascading_gridlock و crossover_showcase تعداد گام‌هایش به بیش از ۱۰۰۰ رسید، در حالی که جواب را در گام ۱۲ و ۴۸ پیدا کرده بود. دلیل این است که در این سناریوها دو بهینه‌ی هم‌هزینه وجود دارد. HC بعد از رسیدن به بهینه، با حرکت‌های افقی بین وضعیت‌های هم‌هزینه جابه‌جا می‌شود تا سقف max_sideways = 100 تمام شود، و این اتفاق در هر ۱۰ restart تکرار می‌شود.

- Simulated Annealing در همه‌ی سناریوها به بهینه رسید، ولی همیشه تا سقف ۳۰۰۰ گام ادامه داد. با α = 0.999، رسیدن دما از ۱۲۰ به ۰.۰۰۱ حدود ۱۱۷۰۰ گام لازم دارد، پس سقف max_iterations زودتر اجرا را قطع می‌کند. SA در هر گام فقط یک همسایه‌ی تصادفی را بررسی می‌کند، به همین دلیل دیرتر از بقیه به جواب می‌رسد.

- Genetic Algorithm کندترین بود (۱۳۹ تا ۱۸۱ میلی‌ثانیه)، با اینکه در ۱ تا ۱۲ نسل اول به بهینه رسید. دلیلش این است که در هر نسل ۵۰ عضو ساخته و ارزیابی می‌شوند و الگوریتم همیشه تا ۱۲۰ نسل ادامه می‌دهد، چون شرط توقف زودهنگام ندارد.

۵. نمودار همگرایی و تحلیل رفتار روی دشت‌ها، قله‌ها و دره‌ها



محور افقی شماره‌ی گام و محور عمودی بهترین زمان سفری است که الگوریتم تا آن لحظه پیدا کرده. چون همیشه بهترین تا الانرسم می‌شود، منحنی هیچ‌وقت بالا نمی‌رود. هر بخش صاف یعنی الگوریتم در آن گام‌ها جواب بهتری پیدا نکرده، و هر پله‌ی پایین‌رونده یعنی یک بهبود. خط Local Beam Search خیلی کوتاه است ۱۲ گام دیده نمیشود درست .

Hill Climbing: منحنی در چند گام اول به‌سرعت پایین می‌آید و در گام تقریبا ۱۰ به بهینه می‌رسد، چون الگوریتم حریصانه همیشه به بهترین همسایه می‌رود. اهمیت restart را این آزمایش نشان می‌دهد: وقتی HC بدون restart و بدون حرکت افقی اجرا شد، در rush_hour روی ۴۵.۰۱، در dual_peak_rush_hour روی ۳۶.۰۰ و در crossover_showcase روی ۳۲.۰۰ متوقف شد. یعنی از حالت شروع پیش‌فرض مستقیم به یک دره‌ی محلی می‌رود که هیچ همسایه‌ای از آن بهتر نیست. فقط با شروع مجدد از نقاط تصادفی به دره‌ی عمیق‌تر می‌رسد.

Simulated Annealing: منحنی SA چند بخش صاف طولانی دارد که هر کدام با یک پله‌ی پایین‌رونده تمام می‌شوند -> این الگو یعنی SA مدتی در یک دره‌ی محلی بوده، ولی چون حرکت‌های بدتر را می‌پذیرد، توانسته از قله ی اطراف آن دره عبور کند و به دره‌ی عمیق‌تری برسد. HC بدون restart چنین کاری نمی‌تواند بکند. در این اجرا آخرین بهبود در گام ۲۴۴ بود و منحنی تا گام ۳۰۰۰ صاف ماند. یعنی بیش از ۹۰٪ زمان اجرا صرف گام‌هایی شد که جواب را بهتر نکردند.

Local Beam Search: منحنی Beam در گام ۲ به بهینه می‌رسد. چون ۸ وضعیت موازی از نقاط مختلف فضا شروع می‌کنند، احتمال اینکه یکی از آن‌ها از ابتدا نزدیک دره‌ی سراسری باشد زیاد است، و استخر مشترک باعث می‌شود بقیه‌ی پرتوها هم سریع به آن ناحیه جذب شوند. محدودیت same_route_limit جلوی جمع شدن همه‌ی پرتوها روی یک مسیر را می‌گیرد، تا الگوریتم روی یک دشت یا یک دره‌ی محلی به‌طور کامل گیر نکند.

Genetic Algorithm: منحنی GA از همان نسل‌های اول نزدیک بهینه است و در نسل ۲ به آن می‌رسد. دلیلش این است که جمعیت اولیه‌ی ۵۰ عضوی و تصادفی بخش بزرگی از ۱۵۶ وضعیت این نقشه را پوشش می‌دهد. crossover می‌تواند بخش‌هایی از دو مسیر خوب را ترکیب کند و با یک پرش بزرگ از روی دره‌ها و دشت‌ها رد شود، که با حرکت‌های کوچک همسایگی ممکن نیست. به لطف elitism بهترین عضو هیچ‌وقت از دست نمی‌رود، پس منحنی هرگز بالا نمی‌رود.

-> HC در دره‌های محلی گیر می‌کند و به restart وابسته است، و روی دشت‌ها (حتی دشت بهینه) گام‌های اضافه برمی‌دارد. SA با پذیرش کنترل‌شده‌ی حرکت‌های بد از دره‌ها فرار می‌کند، ولی کند است و بیشتر زمانش را بعد از رسیدن به جواب هدر می‌دهد. Beam و GA با جستجوی هم‌زمان در چند نقطه از فضا کمتر تحت تأثیر دره‌ها و دشت‌های محلی قرار می‌گیرند. Beam از نظر زمان اجرا بهترین بود و GA به‌خاطر هزینه‌ی بالای هر نسل کندترین.

۶. پاسخ پرسش‌های تحلیلی

پرسش ۲: جستجوی محلی در برابر جستجوی تدریجی

در این مسئله هزینه‌ی هر یال به لحظه‌ی رسیدن به آن یال بستگی دارد، و این لحظه به کل مسیر قبلی و به ساعت حرکت بستگی دارد. پس دو تصمیم کدام مسیر و چه ساعتی به هم گره خورده‌اند. یک مسیر که در ۸:۰۰ بد است ممکن است در ۸:۴۵ بهترین باشد (همان‌طور که در rush_hour دیدیم). جستجوی تدریجی مثل A* یا Dijkstra مسیر را گره به گره می‌سازد و ساعت حرکت باید از قبل ثابت شده باشد. برای بهینه کردن ساعت هم باید جستجو را برای هر ساعت جدا تکرار کرد.

در مدل پیکربندی کامل هر وضعیت یک جواب کامل و معتبر است. شبیه‌ساز می‌تواند آن را یکجا ارزیابی کند، و مسیر و ساعت با یک عملگر همسایگی واحد هم‌زمان تغییر می‌کنند.
چه زمانی جستجوی تدریجی (مثل Time-Dependent Dijkstra) دچار مشکل می‌شود؟
وقتی ویژگی FIFO برقرار نباشد، یعنی دیرتر وارد یال شدن باعث شود زودتر از آن خارج شویم. در این حالت اصل بهینگی زیرمسیرها، که Dijkstra بر آن تکیه دارد، دیگر درست نیست. ممکن است رسیدن دیرتر به یک گره میانی به جواب بهتری برسد، و الگوریتم آن را از دست می‌دهد وقتی ساعت حرکت هم متغیر است باید برای هر ساعت مجاز یک بار اجرا شود. اگر گام زمانی ریز یا بازه بزرگ باشد، یا ساعت پیوسته باشد، این کار پرهزینه یا ناممکن می‌شود.
یا وقتی هزینه جمع‌پذیر نیست یا قیدهای سراسری روی کل مسیر وجود دارد، مثل جریمه برای تعداد گردش‌ها، محدودیت روی کل طول مسیر، یا چند هدف هم‌زمان (زمان و هزینه‌ی سوخت). Dijkstra فقط برای هزینه‌های جمع‌پذیر یال به یال تضمین دارد، ولی جستجوی محلی هر تابع هدفی را روی کل مسیر می‌پذیرد.

در شبکه‌های خیلی بزرگ با به‌روزرسانی مداوم ترافیک، جستجوی تدریجی باید هر بار از اول اجرا شود، ولی جستجوی محلی می‌تواند از جواب قبلی شروع کند و آن را اصلاح کند.
البته اگر FIFO برقرار باشد و فقط یک ساعت حرکت ثابت داشته باشیم، جواب دقیق را با تضمین و سریع پیدا می‌کند. مزیت جستجوی محلی در بهینه‌سازی هم‌زمان مسیر و ساعت است

پرسش ۳: Local Beam Search در برابر Genetic Algorithm

مزیت crossover نسبت به انتخاب حریصانه در Beam:
در Beam Search، تنها راه ارتباط بین پرتوها انتخاب است: پرتوهای خوب همسایه‌های بیشتری در K تای برتر دارند و جای پرتوهای بد را می‌گیرند. ولی هر وضعیت جدید باز هم یک همسایه‌ی یک وضعیت قبلی است، یعنی حداکثر یک تکه‌ی کوچک از مسیر (در کد ما حداکثر ۳ یال) یا ۵ دقیقه از ساعت عوض شده است. اطلاعات خوب یک پرتو هیچ‌وقت با اطلاعات خوب پرتو دیگر ترکیب نمی‌شود.

crossover در GA دقیقا همین کار را می‌کند. ابتدای مسیر یک والد را با انتهای مسیر والد دیگر در یک گره مشترک ترکیب می‌کند، و ساعت حرکت فرزند را از ساعت دو والد می‌سازد. اگر یک والد نیمه‌ی اول خوبی دارد و دیگری نیمه‌ی دوم خوبی، فرزند می‌تواند هر دو را داشته باشد پس GA علاوه بر انتخاب، ترکیب بلوک‌های سازنده‌ی خوب را هم دارد که Beam ندارد.

آیا در Beam دچار Population Collapse شدیم و چطور تنوع را حفظ کردیم؟
بله. بدون محدودیت، بهترین K وضعیت در استخر معمولا یک مسیر با چند ساعت حرکت نزدیک به هم هستند چون این‌ها همسایه‌ی هم‌اند و هزینه‌ی نزدیکی دارند. آزمایش روی cascading_gridlock با ۱۰ seed این را نشان داد:

same_route_limit	میانگین تعداد مسیر متمایز در پرتو پایانی (از ۸)
بدون محدودیت (100)	2.0
2	4.0

یعنی بدون محدودیت ۸ پرتو فقط روی ۲ مسیر جمع شده بودند و جستجو عملا به یکی دو Hill Climbing تبدیل شده بود. با محدودیت ۲، تنوع دو برابر شد. حالت حدی collapse، یعنی K = 1، دقیقاً همان HC است روش‌های حفظ تنوع که استفاده کردیم یا می‌شود استفاده کرد:
- محدودیت تعداد پرتو برای هر مسیر که ما استفاده کردیم
- شروع از وضعیت‌های تصادفی متنوع که پیاده شده است.
- Stochastic Beam Search: انتخاب K پرتو با احتمالی متناسب با کیفیت، به جای K تای برتر.
- تزریق چند وضعیت تصادفی جدید به پرتو در هر چند تکرار.

پرسش ۴: دشت‌ها و حرکت‌های افقی

چه سناریوهای واقعی باعث دشت یا وضعیت‌های تقریبا هم‌هزینه می‌شوند؟
- ساعت‌های دور از اوج ترافیک: وقتی ضریب ترافیک C_e(t) تقریباً صفر است، حرکت ساعت ۷:۰۰ و ۷:۰۵ دقیقا یک زمان سفر دارند. پس همه‌ی حرکت‌های زمانی هزینه را عوض نمی‌کنند و یک دشت در بعد زمان ایجاد می‌شود. در آزمایش ما، وقتی سناریوی flat_plateau روی نقشه‌ای اجرا شد که هیچ یال شلوغی نداشت، تمام فضای حالت یک دشت کامل بود.
- مسیرهای موازی هم‌ارز: دو خیابان موازی با زمان پایه‌ی یکسان و ترافیک مشابه، یا دو مسیر که فقط در یک گره‌ی میانی فرق دارند. در cascading_gridlock و crossover_showcase دیدیم که دو مسیر C2→B2 و C2→T2 دقیقاً هزینه‌ی برابر ۳۱.۰۲ داشتند.
- شلوغی روی یال‌هایی که در مسیر نیستند: تغییر ساعت حرکت فقط روی یال‌های شلوغ مسیر اثر دارد. اگر مسیر از هیچ یال شلوغی عبور نکند، تغییر ساعت بی‌اثر است.
- شانه: در دنباله‌ی منحنی گوسی، ترافیک خیلی آرام تغییر می‌کند. اختلاف هزینه‌ی همسایه‌ها خیلی کوچک است و الگوریتم آن را عملاً صاف می‌بیند.

تأثیر max_sideways روی زمان اجرا و کیفیت:

آزمایش ۱: یک منظره‌ی کاملاً صاف (هیچ یال شلوغی وجود ندارد)، بدون seed ،۱۰ restart:

max_sideways	زمان سفر		تعداد گام‌زمان اجرا (ms)
0	30.00	0	0.1
10	30.00	10	0.8
100	30.00	100	6.7

آزمایش ۲: crossover_showcase (دو بهینه‌ی هم‌هزینه) با seed ۱۰ ،۱۰ restart:

max_sideways		موفقیت (از ۱۰)	تعداد گام‌زمان اجرا (ms)
0	10	43	5.0
10	10	159	12.9
100	10	1155	85.8

روی یک دشت کامل، الگوریتم دقیقا به اندازه‌ی max_sideways قدم افقی برمی‌دارد ، در crossover_showcase هم با افزایش max_sideways از ۰ به ۱۰۰، تعداد گام‌ها حدود ۲۷ برابر و زمان اجرا حدود ۱۷ برابر شد، در حالی که موفقیت در همه‌ی حالت‌ها ۱۰ از ۱۰ بود. دلیلش این است که HC بعد از رسیدن به بهینه، بین دو بهینه‌ی هم‌هزینه رفت و برگشت می‌کند تا سهمیه‌ی حرکت افقی‌اش تمام شود.

نتیجه: حرکت افقی فقط وقتی مفید است که دشت به یک سراشیبی ختم شود اگر دشت واقعا صاف باشد یا خودِ بهینه باشد، حرکت افقی فقط زمان را هدر می‌دهد. در نقشه‌های این تمرین restart برای فرار از دره‌ها کافی بود و max_sideways بزرگ فقط هزینه داشت. یک مقدار متوسط تعادل بهتری بین فرار از شانه‌ها و هزینه‌ی زمانی دارد.

پرسش ۵: برنامه‌ی سرد کردن در Simulated Annealing

ریسک دمای اولیه‌ی خیلی بالا یا خیلی پایین:
احتمال پذیرش حرکت بدتر $P = \exp(-\Delta E/T)$ است.

اگر T_0 خیلی بالا باشد، P برای تقریبا همه‌ی حرکت‌ها نزدیک ۱ است. الگوریتم در ابتدای کار یک قدم زدن تصادفی در فضای حالت است و تا وقتی دما پایین بیاید، تعداد زیادی گام بدون پیشرفت هدر می‌رود. در آزمایش ما با $T_0 = 10000$، الگوریتم ۱۴۸۶ حرکت بدتر را پذیرفت (در برابر ۱۱۴۲ برای $T_0 = 120$). اگر سقف تکرار کم باشد، ممکن است جستجو قبل از رسیدن به فاز بهره‌برداری تمام شود...
- اگر T_0 خیلی پایین باشد، P از همان ابتدا نزدیک صفر است و SA تقریبا هیچ حرکت بدی را نمی‌پذیرد. یعنی عملاً یک Hill Climbing تصادفی است و در اولین دره‌ی محلی گیر می‌کند. در آزمایش ما با $T_0 = 1$ فقط ۹۶ حرکت بدتر پذیرفته شد. چون اختلاف هزینه‌ها در این نقشه کوچک است، باز هم به بهینه رسید. ولی در منظره‌ای با دره‌های عمیق‌تر، این دما برای فرار کافی نیست.

تأثیر α روی رفتار جستجو (rush_hour، $T_0 = 120$، ۱۰ seed):

α	تعداد گام‌حرکت بدتر پذیرفته‌شده		موفقیت (از ۱۰)	میانگین زمان سفر
0.5	17	2	0	48.19
0.9	112	12	0	44.81
0.992	1457	180	9	41.94
0.999	3000	1142	10	41.77

با $\alpha = 0.5$ (سردسازی ناگهانی، Quenching)، دما در هر گام نصف می‌شود و بعد از ۱۷ گام به min_temp می‌رسد. در این مدت فقط ۲ حرکت بدتر پذیرفته شد. الگوریتم هیچ‌وقت به بهینه نرسید و میانگین جوابش (۴۸.۱۹) حتی از HC ساده بدون restart (۴۵.۰۱) بدتر بود، چون فرصت کافی برای پایین آمدن هم نداشت. $\alpha = 0.9$ هم وضع مشابهی داشت.

با $\alpha = 0.999$ دما خیلی آهسته پایین می‌آید. الگوریتم مدت طولانی در فاز کاوش می‌ماند، بیش از هزار حرکت بدتر را می‌پذیرد و در هر ۱۰ اجرا به بهینه می‌رسد. هزینه‌اش زمان اجرای بیشتر است، و در این حالت سقف ۳۰۰۰ تکرار زودتر از رسیدن دما به min_temp اجرا را قطع کرد (برای رسیدن از ۱۲۰ به ۰.۰۰۱ حدود ۱۱۷۰۰ گام لازم است).

نتیجه: سرد کردن آهسته به الگوریتم فرصت می‌دهد ابتدا ناحیه‌ی خوب فضای حالت را پیدا کند و بعد در آن دقیق شود. سرد کردن ناگهانی SA را به یک جستجوی حریصانه‌ی کوتاه تبدیل می‌کند که در اولین دره‌ی محلی می‌ماند.